

Save-load

```
#include "../include/dsl.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
//
// Funções para guardar o jogo
//
```

```
// Função que traduz o número do naipe que está na memória para a letra
correspondente para escrever o ficheiro
```

```
char obter_naipe_char(int n) {
    const char naipes[] = "SHDC";
    return naipes[n];
```

→ resolve a letra correspondente ao naipe que durante o jogo é guardado em números

```
// Função que escreve todas as cartas de uma única pilha numa linha de texto
```

```
void gravar_pilha(FILE *f, Pilha p) {
```

```
    int i = 0;
```

```
    const char *vals[] = {"", "A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"};
```

Traduz os valores

```
    while (i < p.quantidade) {
```

→ ciclo que percorre todas as cartas

```
        fprintf(f, "%s%c", vals[p.cartas[i].num],
```

```
                obter_naipe_char(p.cartas[i].naipe));
```

→ O "%s%c" junta o número com o naipe

```
        if (i < p.quantidade - 1) fprintf(f, " ");
```

→ Se ainda não estiver na última carta, insere um espaço para separar as cartas

```
        i++;
```

```
    }
    fprintf(f, "\n");
```

→ Quando acaba de ler

```
// Cria o ficheiro 'save' final
```

```
int guardar_jogo(EstadoJogo *e, const char *nome_paciencia, const char
*ficheiro_save) {
```

```
    FILE *f = fopen(ficheiro_save, "w");
```

```
    int i = 0;
```

```
    if (f == NULL) return 0;
```

→ Se não conseguirmos retorna 0

```
    fprintf(f, "JOGO %s\n", nome_paciencia);
```

```
    while (i < e->total_pilhas) {
```

```
        gravar_pilha(f, e->pilhas[i]);
```

```
        i++;
```

→ Ciclo que grava todas as pilhas

```
    }
```

```
    fclose(f);
```

```
    return 1;
```

```
//
```

```
// Funções para carregar o jogo
```

```
//
```

```
// Função que converte o naipe em números
```

```
int converter_naipe(char n) {
```

```
    if (n == 'H') return 1;
```

```
    if (n == 'D') return 2;
```

```
    if (n == 'C') return 3;
```

```
    return 0;
```

É chamada para todas as pilhas

Escreve diretamente no ficheiro


```

}

// Função que converte as cartas especiais em números
int converter_valor(char *v) {
    if (v[0] == 'A') return 1;
    if (v[0] == 'J') return 11;
    if (v[0] == 'Q') return 12;
    if (v[0] == 'K') return 13;
    return atoi(v); // ASCII to int
}

```

Comente as restantes variáveis

```

// Pega numa única linha de texto e transforma-a numa coluna de cartas
void processar_linha_save(Pilha *p, char *linha) {
    free(p->cartas); // Deixa zona as cartas criadas quando o jogo foi carregado
    p->cartas = NULL;
    p->quantidade = 0;
    char *token = strtok(linha, " \n");
    while (token != NULL) {
        p->cartas = realloc(p->cartas, (p->quantidade + 1) * sizeof(Carta));
        int len = strlen(token);
        p->cartas[p->quantidade].naipe = converter_naipe(token[len-1]);
        p->cartas[p->quantidade].num = converter_valor(token);
        p->quantidade++;
        token = strtok(NULL, " \n");
    }
}

```

String tokenizem → Conta a frase sempre que vê um espaço ou \n

Altera espaço para +1 carta

obtem o tamanho da palavra

Salvamos que a ultima palavra é sempre o naipe

Comute o valor

Conta a próxima palavra

```

// Função que restaura o jogo inteiro que está no save.txt
int carregar_save(EstadoJogo *e, const char *ficheiro, char *caminho_lido) {
    FILE *f = fopen(ficheiro, "r"); // Abre o ficheiro em modo leitura (r)
    if (!f) return 0; // Se não conseguir da erro
    char linha[512];
    if (fgets(linha, sizeof(linha), f) && sscanf(linha, "JOGO %s",
        caminho_lido) == 1) {
        carregar_paciencia(caminho_lido, e); // Carrega o jogo
        int p = 0; // Coloca em linha comprimento origem
        while (fgets(linha, sizeof(linha), f) && p < e->total_pilhas) {
            processar_linha_save(&e->pilhas[p], linha);
            p++;
        }
        fclose(f);
        return 1;
    }
}

```

Carrega o jogo

Coloca em linha comprimento origem

Processa a linha

Avança para a próxima

Fecha

Ê as cartas e envia para pilhas e envia para furos auxiliares